

Adventure Sample Loop Structure

현재 Unity 프로젝트 기준 샘플 어드벤처 게임 루프 구조 문서

Purpose

현재 목표는 GridChunk 랜덤맵을 기반으로 한 1차 플레이 루프를 만드는 것이다.

```
TitleScreen.StartGame()
-> SceneFlowManager가 Game 씬 로드
-> AdventureRunManager.StartRun()
-> MapManager.GenerateMap()
-> GridChunkRuntimeGenerator가 랜덤맵 생성
-> GeneratedMapInfo에서 시작/목표 위치 획득
-> Player 생성 또는 배치
-> Camera / HUD / Input 연결
-> GameManager.Playing
-> 탐험 / 전투 / 아이템 / 함정 / 대화
-> EndLine 도달 시 클리어 또는 다음 맵
```

Manager Responsibility

Manager	Responsibility
Managers	전역 접근 루트. Game, SceneFlow, UI, Map, AdventureRun, Player를 연결한다.
GameManager	게임 상태를 관리한다. 현재는 Playing, Paused, GameOver 중심이다.
SceneFlowManager	타이틀/게임 씬 전환, 로딩 오버레이, 씬 리셋을 담당한다.
UIManager	HUD, 메뉴, 로딩, 알림, 대화 UI의 진입점을 관리한다.
PlayerManager	플레이어 입력, 인벤토리, 장비 데이터를 보관한다.
MapManager	GridChunk 랜덤맵 생성/삭제와 현재 GeneratedMapInfo 보관을 담당한다.
AdventureRunManager	한 판의 시작, 진행 상태, 클리어, 실패, 정리를 담당한다.

Current Implemented Flow

```
TitleScreen
|
v
SceneFlowManager.StartGame()
|
v
TransitionTo(Game, InGame)
|
v
Managers.Instance.AdventureRun.StartRun()
|
v
AdventureRunManager
|
+-- GameManager.Reset()
|
+-- MapManager.GenerateMap()
|
|
v
GridChunkRuntimeGenerator.Generate(settings)
```

```
|
+-- GridChunkDatabase에서 청크 후보 선택
+-- GeneratedGridMap 루트 생성
+-- Start / Content / Side Branch / End 청크 배치
+-- SpawnKind 기반 몬스터, 아이템, 환경 배치
+-- GeneratedMapInfo 생성
```

현재 `AdventureRunManager.StartRun()` 은 맵 생성까지 담당한다. 플레이어 생성, 카메라 연결, GoalTrigger 연결은 다음 단계에서 추가한다.

Runtime Map Generation

Settings Asset

```
GridMapGenerationSettings1.asset
+-- GridChunkDatabase
+-- contentChunkCount
+-- maxThreeDirCount
+-- useRandomSeed
+-- randomSeed
+-- GridSpawnPrefabSet
```

Generated Result

```
GeneratedGridMap
+-- GridChunk prefab instances
+-- Spawned content objects
+-- GeneratedMapInfo
+-- playerSpawnPosition
+-- endPosition
```

Target Adventure Loop

```
Run Start
|
v
Random Map Generated
|
v
Player Spawned
|
v
Camera / HUD / Input Bound
|
v
Exploration
|
+-- Combat
+-- Item Pickup
+-- Equipment
+-- Trap Avoidance
+-- NPC Dialogue
|
v
Goal Reached or Player Dead
|
+-- Goal Reached -> Run Clear / Next Map
+-- Player Dead -> GameOver
```

Implementation Roadmap

Done Step 1. Run Flow Entry

- `SceneFlowManager` 가 게임 실행 로드 후 `AdventureRunManager.StartRun()` 을 호출한다.
- 직접 `MapManager.GenerateMap()` 을 호출하지 않고 한 판 진행 관리자를 거치도록 정리했다.

Done Step 2. Manager Structure

- `Assets/Scripts/Managers/AdventureRunManager/AdventureRunManager.cs` 추가.
- `Managers` 에 `AdventureRunManager` 참조 추가.
- `MapManager` 는 맵 생성/삭제 책임만 유지한다.

Next Step 3. Player Spawn

- `AdventureRunManager` 에 `playerPrefab` 필드를 추가한다.
- `GeneratedMapInfo.playerSpawnPosition` 에 Player를 생성하거나 기존 Player를 이동시킨다.
- 생성된 Player를 현재 런의 플레이어로 보관한다.

Next Step 4. Camera Binding

- `CameraFollow2D` 를 만들거나 `Cinemachine Virtual Camera`를 사용한다.
- 런타임에 생성된 Player를 Follow 타겟으로 연결한다.

Next Step 5. Goal Trigger

- `GoalTrigger` 또는 `EndTrigger` 를 추가한다.
- `GeneratedMapInfo.endPosition` 에 Goal prefab을 생성한다.
- Player가 닿으면 `AdventureRunManager.EndRun()` 또는 `HandleGoalReached()` 를 호출한다.

Next Step 6. Game State Expansion

- `GameManager` 에 `Clear` 또는 `RunComplete` 상태를 추가할지 결정한다.
- `SetPlaying()`, `TriggerClear()` 같은 명확한 상태 전환 메서드를 추가한다.

Next Step 7. Death Flow

- 현재 Player death는 직접 `GameManager.TriggerGameOver()` 로 흐른다.
- 이후에는 `AdventureRunManager.FailRun()` 을 거쳐 런 정리 후 `GameOver`로 가게 바꾼다.

Recommended Final Shape

```
Managers
├─ GameManager
├─ SceneFlowManager
├─ UIManager
├─ PlayerManager
├─ MapManager
└─ AdventureRunManager

AdventureRunManager
├─ StartRun()
├─ SpawnPlayer()
├─ BindCamera()
├─ BindHUD()
├─ HandleGoalReached()
├─ HandlePlayerDead()
└─ ClearRun()

MapManager
├─ GenerateMap()
├─ ClearMap()
└─ CurrentMapInfo
```

Notes

- `EditorWindow` 는 설정 편집과 미리보기 생성 도구로만 사용한다.

- 실제 맵 생성 알고리즘은 `GridChunkRuntimeGenerator` 하나로 통합했다.
- `GridMapGenerationSettings1.asset` 을 기준으로 Editor 미리보기와 Runtime 생성이 같은 값을 사용한다.
- 다음 구현의 핵심은 `AdventureRunManager` 가 Map, Player, Camera, UI를 연결하는 오케스트레이터가 되는 것이다.